



Projet Microcontrôleur

BACHELOR IV - MICROTECHNIQUE

LIMITEUR DE TEMPÉRATURE

PROFESSEUR : ALEXANDRE SCHMID

VRAY Alexandre
LJUNGBERG Oliver

Groupe 52
Printemps 2021

Introduction : Dans le cadre du projet de microcontrôleur, nécessitant - entre autres - l'utilisation du capteur de température, nous avons décidé de créer un limiteur de température pour bassin à poissons, ouvrant progressivement une valve d'eau froide lorsque la température devient trop chaude, et qui sonne lorsque le système devient tellement chaud qu'il devient incontrôlable.

Description générale : Notre programme va utiliser le capteur de température (1-wire dallas), l'encodeur, le moteur servo Futaba S3003 codé en angle, l'écran LCD et enfin le buzzer, comme périphériques externes. Le but du programme est de pouvoir choisir la température limite qu'on ne veut pas dépasser dans le bassin, et à partir de combien de degrés au dessus de cette température le système devient incontrôlable et lance l'alarme. On peut également choisir l'unité de la température. L'encodeur sert donc à l'utilisateur pour modifier les réglages, l'écran LCD à afficher les paramètres qui nous intéressent, le moteur servo à ouvrir et fermer la valve d'eau froide, le capteur de température pour prélever la température et le buzzer à indiquer lorsque le système est en surchauffe.

1

1.1 Mode d'emploi

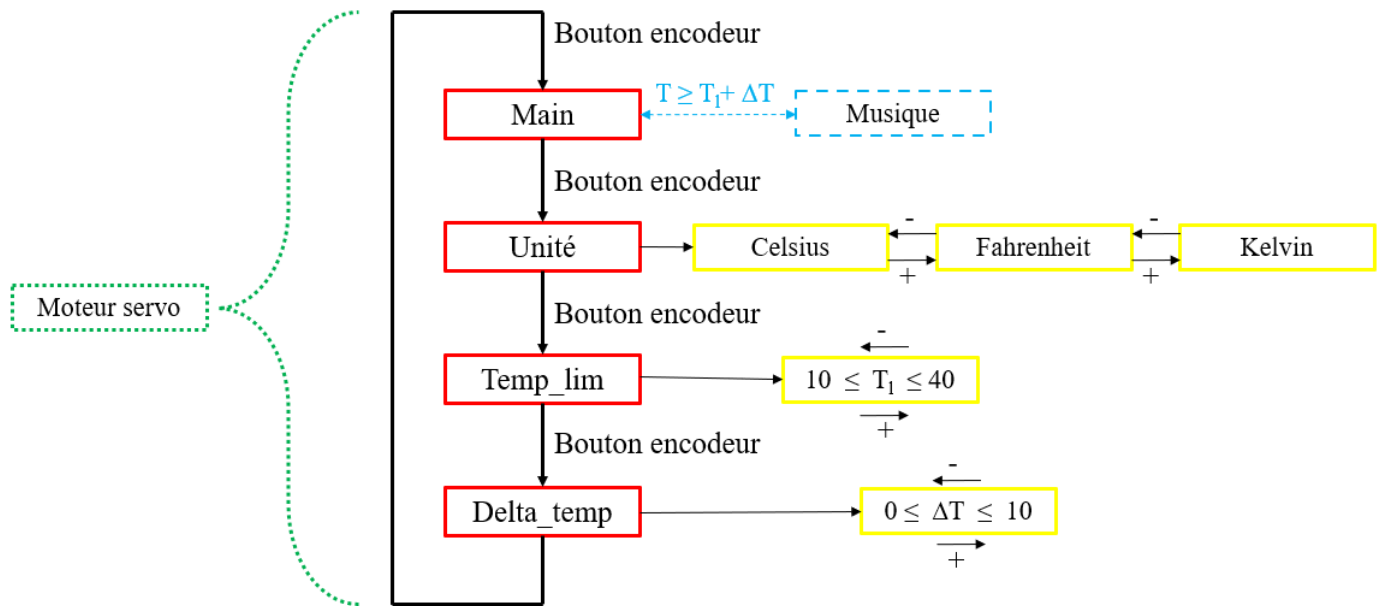


FIGURE 1 – Schéma mode d'emploi

Main : Lorsqu'on est dans le *Main*, l'écran LCD affiche la température réelle et la température limite. Si la température réelle dépasse la température limite de ΔT , la musique "Au feu les pompiers" se met en route.

Lorsqu'on appuie sur le bouton de l'encodeur depuis le *Main* (que la musique soit en route ou non), on passe dans le réglage *unité* (et la musique s'arrête si elle était allumée).

Unité (Réglage 1) : L'écran LCD affiche l'unité de température que l'on utilise. Avec l'encodeur angulaire, on peut faire varier l'unité entre Celsius, Fahrenheit et Kelvin.

Lorsqu'on appuie sur le bouton, on passe dans le réglage *Temp_lim*.

Temp_lim (Réglage 2) : L'écran LCD affiche la température limite seulement. Elle peut être réglée entre 10°C et 40°C. L'incrément et la décrémentation de l'encodeur sont bloquées aux extrémités. La température limite est initialement fixée à 25°C.

On passe dans le réglage *Delta_temp* lorsqu'on appuie sur le bouton de l'encodeur.

Delta_temp (Réglage 3) : L'écran LCD affiche la différence de température entre la température limite et la température de surchauffe. Elle peut être réglée entre 0°C et 10°C de la même manière que pour le réglage de la température limite.

Lorsqu'on appuie sur le bouton de l'encodeur, on retourne dans le *Main*.

Moteur servo : Le moteur servo a un angle d'ouverture variant entre 0° et 90°. Il est réglé automatiquement lors d'interruptions, atteignant un angle de 90° lorsque la température réelle dépasse la température de

surchauffe, atteignant un angle de 0° lorsque la température réelle est plus faible que la température limite, et variant linéairement entre 0° et 90° si la température réelle se trouve entre la température limite et la température de surchauffe.

1.2 Description technique

Ports et périphériques : Les périphériques utilisés sont branchés sur les ports suivants :

- **Capteur de température :** Port B, Pin5
- **Moteur servo :** Port D, Pin 4
- **Encodeur :** Port E, Pins 4 et 6
- **Buzzer :** Port E, Pin 2
- **Écran LCD :** Connecteur LCD

Interfaces d'acquisition et d'affichage : Le seul périphérique d'interface de l'utilisateur vers le microcontrôleur est l'encodeur. Les périphériques d'interface du microcontrôleur vers l'utilisateur sont l'écran LCD et le Buzzer.

Interruptions : Nous avons des interruptions sur deux timers différents : le timer 0 pour le moteur servo, et le timer 1 pour vérifier l'état du bouton de l'encodeur. Le timer 0 est réglé avec le prescaler à 6, et le timer 1 avec le prescaler à 1. Les deux timers ont donc une interruption toutes les 16.384 ms. Le moteur va recalculer s'il a la bonne ouverture, et bouger en fonction.

1.3 Fonctionnement du programme

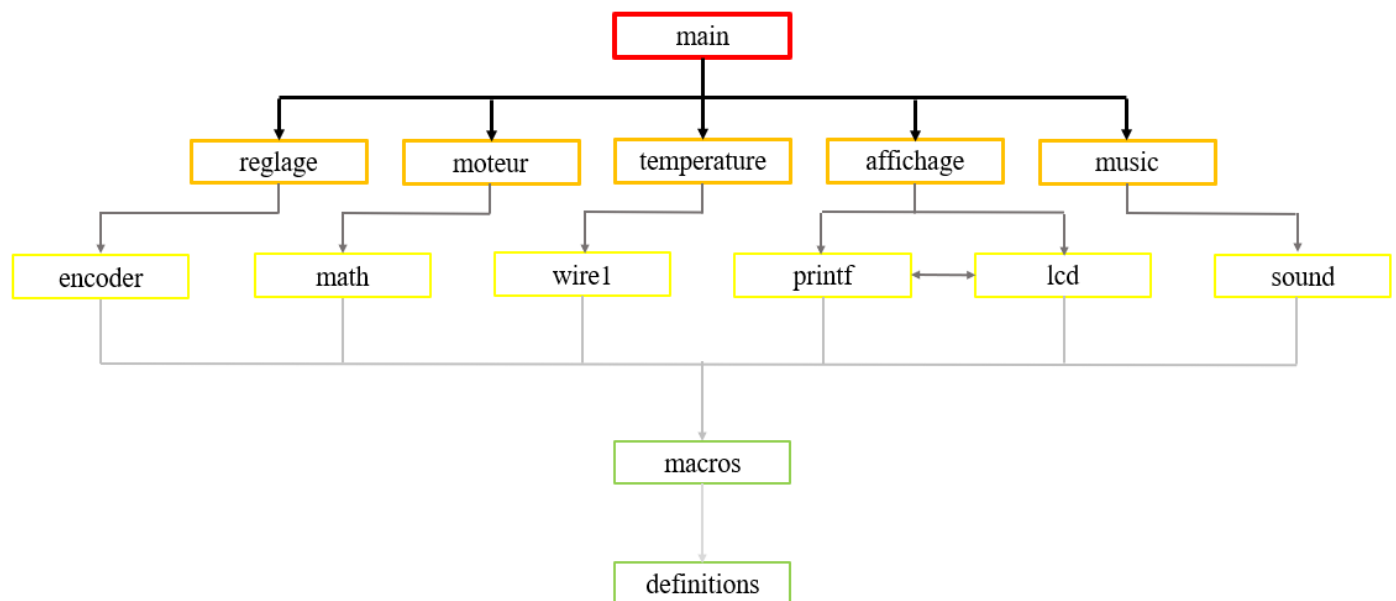


FIGURE 2 – Schéma de dépendances des fichiers

Le fichier *main.asm* est directement dépendant de cinq fichiers indépendants :

- *reglage.asm*, qui cherche à savoir dans quel réglage on se trouve, et qui dépend de *encoder.asm*, pour connaître les actions réalisées par l'encodeur.

- *moteur.asm*, qui règle l'angle du moteur servo, et qui dépend de *math.asm* pour linéariser l'angle d'ouverture.
- *temperature.asm*, qui donne la température réelle captée, et qui dépend de *wire1.asm*.
- *affichage.asm*, qui s'occupe de l'écran LCD, et qui dépend de *printf.asm* et de *lcd.asm*, qui possèdent les sous-routines et macros liées à l'affichage.
- *music.asm*, qui s'occupe de la musique et du buzzer, et qui dépend du fichier *sound.asm*, qui est la librairie détenant toutes les fonctionnalités liées aux notes de musique.

Tous ces 12 fichiers dépendent également du fichier *macros.asm*, et enfin de *definitions.asm* (*macros.asm* y compris).

1.4 Présentation des modules

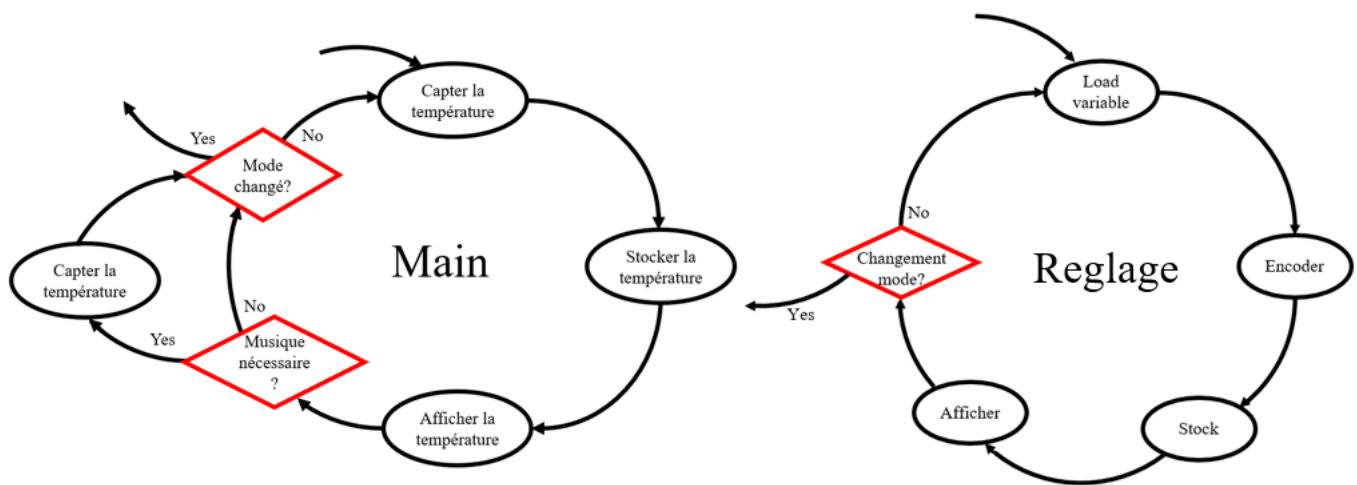


FIGURE 3 – Schéma des fonctions et modules principaux

Lorsqu'on se trouve dans le mode principal (*Main*), le programme va tout d'abord aller chercher les fichiers s'occupant de capter la température réelle, puis la stocker dans une variable (*temp*), et la température réelle ainsi que la température limite (stockée dans *temp_lim*) vont être affichés grâce aux fichiers s'occupant du LCD. Seulement après ces trois étapes, il est possible de changer l'affichage : tout d'abord, on vérifie si on se trouve au dessus de la température de surchauffe. Si c'est le cas, la musique se met en route, et l'écran LCD affiche "Attention! temp trop haute". À la fin de la musique, la température est captée à nouveau. Il est ensuite possible de sortir du mode principal (que la musique soit allumée ou non) et de changer de mode, en appuyant sur le bouton de l'encodeur. Si ce n'est pas le cas, la boucle recommence.

Pour les trois modes secondaires (*Reglage1*, *Reglage2*, *Reglage3*), le fonctionnement est assez similaire : la variable en question (respectivement *temp_unit*, *temp_lim*, et *delta_temp*) est chargée, puis modifiée avec l'encodeur, avant d'être stockée dans le main, puis affichée sur l'écran. Ensuite on ressort du mode par le même moyen que pour le *Main*.

Pour le moteur servo, puisqu'il fonctionne avec les interruptions, on entre dans son module et on en ressort à chaque interruption liée au timer 0.

2 Accès aux périphériques

2.1 Capteur de température

Pour notre projet, le principal périphérique est le capteur de température. Il est nécessaire dans la partie principale de notre programme. Nous allons dans cette section comprendre comment celui-ci est utilisé.

Où stocker la valeur de la température ? Tout d'abord, dans notre programme, la température est nécessaire pour des tests afin de faire varier l'affichage et la position de notre moteur futaba S3003. Nous décidons donc de stocker sa valeur dans la mémoire SRAM. La documentation nous informe que le capteur de température DS18B20 a une résolution programmable pouvant aller jusqu'à 12 bits. Ainsi nous avons 2 bytes de la SRAM pour la variable que nous appellerons "*temp*".

Comment capter la température ? L'utilisation du capteur de température se fait par le standard 1-Wire bus développé par la firme Dallas. Ainsi, nous pouvons accéder à la température selon une transmission 1-Wire. Nous avons décidé de connecter notre capteur de température sur le port B. Sur le module M5, le Dallas 1-Wire est connecté au pin 6. Nous nous intéressons donc au pin 6 du port B.

Nous commençons dans le fichier *temperature.asm* par envoyer un reset pulse afin d'initialiser la transmission one wire entre le maître et l'esclave. Ceci se réalise à l'aide du fichier *wire1.asm*. Juste après, à l'aide de ce même fichier, nous initialisons la conversion de la température au format binaire. En réalisant à nouveau un reset pulse, nous pouvons maintenant lire la température avec la commande *wire1_read*. Nous commençons par les LSB puis les MSB. Finalement, grâce aux registres a1 et a0 (respectivement r19 et r18), nous stockons la valeur qu'ils contiennent dans notre variable *temp*.

Intéressons nous donc à quand cette routine de lecture *tps* est appelée.

Quand demander de capter la température ? Dans notre projet, la température n'est en aucun cas nécessaire durant un mode de réglage. Donc celle-ci n'est appelée que lors de la routine *main* et ainsi de manière cyclique du moment que nous restons dans cette routine *main*.

Mais comment utiliser la température pour la rotation de notre moteur servo Futaba S3003 ?

2.2 Moteur servo Futaba S3003

Le moteur servo Futaba S3003 est le second périphérique le plus important de notre système, il est en quelque sorte le but de celui-ci. Nous cherchons à linéariser la position du moteur si la température *temp* se trouve entre la température limite *temp_lim* et la température maximum *temp_lim + delta_temp*.

Quelle information transmettre au moteur servo ? La position du moteur servo sera codée dans les registres a1 et a0 (respectivement r19 et r18). Notons aussi qu'en se basant sur la datasheet du motor Futaba S3003, une différence d'angle de 90° est réalisée par une différence de temps de 800µs (aussi une différence de 800 dans nos registre a1 et a0). Ainsi dans notre sous routine *moteur*, nous commençons par charger les différentes variables nécessaires, les mettre à la même unité puis enfin nous les comparons à *temp_lim* et *temp_lim + delta_temp*. Trois cas sont possibles :

- 1) $temp < temp_lim$
- 2) $temp_lim < temp < temp_lim + delta_temp$
- 3) $temp > temp_lim + delta_temp$

Dans le cas 1), nous voulons l'angle minimum, ainsi nous paramétrons a1 et a0 à 0. Dans le cas 3) nous voulons l'angle maximum d'où nous chargeons la valeur de 800 dans a1 et a0.

C'est le cas 2) qui nous semble le plus intéressant. Nous savons que $0 < temp - temp_lim < delta_temp$

et nous voulons linéariser $a1$ et $a0$ entre 0 et 800. Donc nous réalisons à l'aide des sous-routines du fichier "*math.asm*" le calcul suivant :

$$a1, a0 = \frac{(temp - temp_lim) \cdot 800}{delta_temp}$$

Intéressons nous à comment transmettre l'information des registres $a1$ et $a0$ au moteur servo.

Comment transmettre l'information au moteur servo ? Nous avons ici possession d'une valeur stockée sur deux registre $a1$ et $a0$, et nous savons que cette valeur est comprise entre 0 et 800. Cependant, pour transmettre une information d'angle 0° au moteur servo, la datasheet nous informe qu'il faut envoyer une pulse de $1520\mu s$. Nous voulons que l'angle varie entre 0° et 90° , ce qui réalise une différence totale de $800\mu s$ ainsi la pulse a une durée comprise dans l'intervalle $1520 - 400 = 1120\mu s < pulse < 1520 + 400 = 1920\mu s$. Tous ceci nous indique que nous devons ajouter un offset de 1120 à nos registres $a1$ et $a0$ pour garantir cet intervalle.

Finalement nous réalisons ce pulse avec une loop qui décrémente $a1, a0$ chaque microseconde (4 cycles d'horloge internes).

Cependant, quand devons nous envoyer ces informations au moteur servo ?

Quand appeler la sous-routine moteur ? Pour un mouvement fluide, le moteur servo doit recevoir une information lui guidant sa position de manière suffisamment fréquente. Ainsi nous avons utilisé un timer d'interruption qui appelle notre sous routine *moteur* toutes les 16.384 ms (timer 0 avec un prescaleur de 256).

2.3 Encodeur angulaire

L'encodeur angulaire est notre périphérique permettant l'interface utilisateur, c'est avec celui-ci que l'utilisateur peut régler les paramètres à sa convenance. Ainsi la sous-routine encodeur n'est utilisée que dans le fichier "*reglages.asm*"

Comment utiliser l'encodeur ? Nous utilisons le fichier "*encoder.asm*" et sa sous routine *encoder* qui modifie directement les registres $a1$ et $a0$ en fonction du mouvement réalisé par l'utilisateur. Celui-ci peut faire varier $a1, a0$ entre $0x00$ et $0xff$. Cependant, dans nos 3 modes de réglages, nous voulons limiter cette plage entre les valeurs suivantes :

mode1 : entre 0 et 2

mode2 : entre $0x28$ et $0x78 \Rightarrow$ respectivement $10^\circ C$ et $40^\circ C$ si nous considérons 2 bits décimaux.

mode3 : entre 0 et $0x50 \Rightarrow 10^\circ C$ en considérant 3 bits décimaux

Pour réaliser ceci, nous vérifions après utilisation de l'encodeur si nous avons dépassé la plage admissible du mode correspondant. Si oui, nous rechargeons $a1$ et $a0$ à la borne la plus proche (borne min ou borne max). La variable à modifier a donc été chargée avant d'appeler la sous-routine encodeur et est à nouveau stockée après nos différentes vérifications.

Quand appeler les sous-routines de réglage ? Nous possédons trois modes de réglages. Chacun possède sa sous-routine de réglage pour des raisons de variables différentes et plages admissibles différentes. Celles-ci sont appelées de manière répétitive tant que nous restons dans le même mode.

Bouton de l'encodeur angulaire : Parlons de l'utilisation du bouton I. C'est celui que nous utilisons pour le changement de mode. Celui-ci étant sur le pin 6 du module, nous n'avons pas pu utiliser les interruptions asynchrones disponibles sur les pins 0 à 4 du portD. Nous avons donc dû trouver une alternative. Celle-ci

consiste en l'utilisation d'un timer d'interruptions qui va vérifier l'état du bouton régulièrement (suffisamment pour qu'on n'ait pas le temps d'appuyer/relâcher : nous avons choisis 16.384 ms avec l'utilisation du timer 1 et un prescaler de 1). Seul si le bouton n'était pas appuyé à la dernière interruption, et l'est devenu à celle-ci permet un changement de mode (on charge le pointeur *z* avec l'adresse du mode correspondant depuis la variable *mode_selec* puis on incrémente *z* afin de stocker l'adresse du nouveau mode°. Seul à la fin de chaque mode, le changement peut s'opérer avec une lecture de l'adresse stockée dans *mode_selec*.

2.4 Affichage LCD

L'affichage LCD est le deuxième périphérique nous permettant l'interface utilisateur. L'encodeur permet à l'utilisateur de communiquer au microcontrôleur, l'affichage LCD est là pour l'inverse : il permet au microcontrôleur de communiquer à l'utilisateur.

Quoi afficher ? Chaque mode du programme a sa propre sous-routine d'affichage en fonction de ce qui est en train de se réaliser. Le main veut afficher la température réelle et la température limite alors que les modes de réglages cherchent à afficher dans quel réglage nous sommes, et la variable à modifier.

Comment afficher ? Nous utilisons les fichiers "*LCD.asm*" et "*printf.asm*" pour être en possession des sous-routines et macros nécessaires à l'affichage. Dans notre programme, nous nous occupons principalement de la conversion de nos données vers une valeur compréhensible par l'utilisateur (une valeur décimale avec la bonne unité). Le réglage 1 permettant la conversion d'unité, c'est uniquement à l'affichage que la différence se fait. Nous vérifions quelle unité est gardée dans la variable *temp_unit* de la SRAM et réalisons la conversion nécessaire des registres a1,a0 ou b1,b0 dans lesquelles nous avons chargé les variables nécessaires au préalable. À l'affichage, nous pouvons préciser combien de bits représentent la partie décimale (ex : 4 pour la température réelle *temp*).

Quand afficher ? Les sous-routines d'affichage sont appelées directement après l'application de chaque mode. C'est à dire juste après avoir lu la température dans le main, ou après une sous-routine de réglage dans les autres modes.

2.5 Buzzer

Le buzzer est un petit plus à notre projet, il permet aussi une communication du microcontrôleur vers l'utilisateur.

Que transmettre au buzzer ? Nous avons décidé de réaliser la musique "Au feu les pompiers", ainsi avec l'aide du fichier "*sound.asm*" nous avons réalisé une look-up table comprenant toutes les notes de notre mélodie. C'est celle-ci que nous voulons transmettre au buzzer.

Comment utiliser le buzzer ? Maintenant que nous avons notre look-up table, nous initialisons le pointeur *z* au début de celle-ci et réalisons une boucle *play* qui permet de jouer (Sur le pin 2 du Port E) la note pointée puis incrémente le pointeur *z* jusqu'à la fin de la table. Nous avons aussi du régler le registre b0 qui détermine durant combien de temps chaque note doit être jouée.

Quand appeler la sous-routine *music* ? Nous ne voulons jouer la mélodie que si la température *temp* dépasse le seuil de *temp_lim* + *delta_temp*. Ainsi, à chaque boucle de la routine main, nous devons passer par la sous-routine *music_on*, qui vérifie cette condition et agit en conséquence.

A Annexes

A.1 Schémas supplémentaires

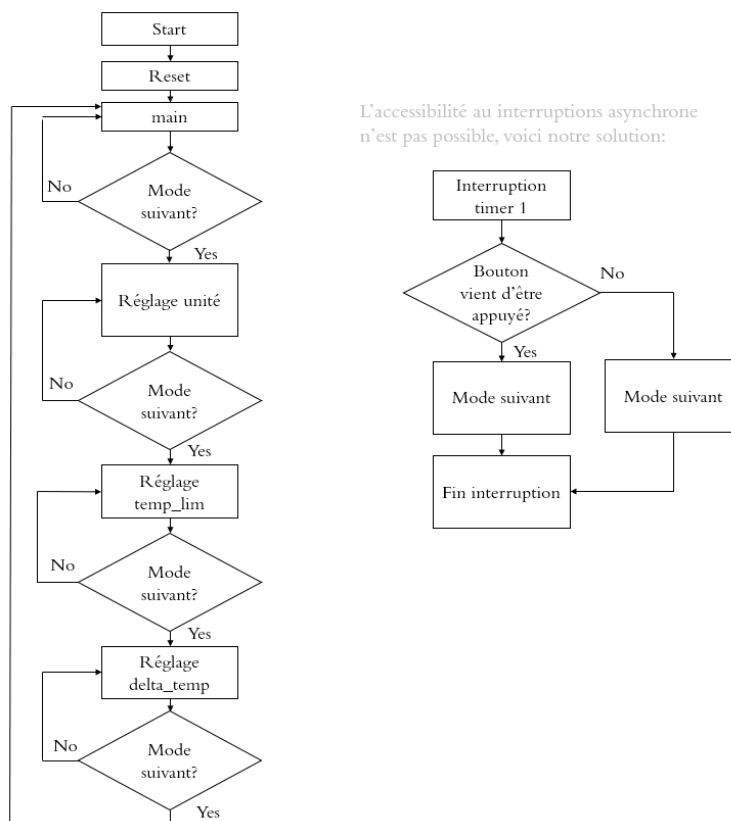


FIGURE 4 – Organigramme général du programme

A.2 Code source